高级语言程序设计大作业实验报告

KeyVizOverlay：实时按键可视化叠加层

姓名: 张译仁
学号: 2211051

课程: 高级语言程序设计
实验: 课程大作业
日期: 2026-03-25
平台: Windows + Win32 + D3D11 + ImGui
工具: Visual Studio 2022 / C++20

摘要

本报告记录 KeyVizOverlay 项目的设计与实现过程。项目目标是实现一个 C++ 图形化叠加层，在录屏、直播或演示时实时显示键盘和鼠标输入状态。实现上使用 Win32 分层窗口承载界面，RawInput 采集输入，D3D11 + Dear ImGui 完成渲染；支持布局预设、透明度与缩放调节以及自定义键位编辑。文中补充了架构图与执行链路图，并配套测试用例说明关键功能验证路径。

1 作业题目

题目： KeyVizOverlay（实时按键可视化叠加层）
目标：

- 在 Windows 桌面环境中实现悬浮式图形化小程序；
- 实时显示常用键盘按键与鼠标按键状态；
- 支持外观调节与布局自定义，满足不同游戏/演示场景；
- 保持低侵入（可点击穿透）与可维护的工程结构。

2 开发软件

- 语言标准：C++20
- IDE / 工程：Visual Studio（`.vcxproj`，Toolset `v145`）
- 图形与窗口：Win32 API + Direct3D 11
- UI 库：Dear ImGui（`imgui_impl_win32` + `imgui_impl_dx11`）
- 输入：Windows RawInput（键盘 + 鼠标）
- 版本管理：Git + GitHub

项目仓库：https://github.com/yirenzhang/KeyVizOverlay.git

3 课题要求

表 1: 课题要求与实现对应

要求项	实现方式
使用 C++ 完成图形化小程序	采用 C++20 + Win32 + D3D11 + ImGui，实现桌面图形化悬浮层。
面向对象设计	按职责拆分 `Application`、`Win32Window`、`D3D11Renderer`、`InputService`、`OverlayUI` 等类。
创新与创意	支持键位实时发光、像素风键帽、`hidehide/showshow` 控制台命令、自定义键位拖拽编辑。
模型部分验证	输入状态模型（按下/释放/按住时长/动画进度）与布局模型（行-键结构、预设文件）均设置了边界约束。
过程可追踪	仓库包含多个阶段提交（feat/fix/refactor），能看到版本迭代过程。

4 主要流程

4.1 整体流程

程序主流程如下：

1. `main.cpp` 创建 `Application` 并进入消息循环；
2. 初始化窗口（透明置顶 + RawInput 注册）与 D3D11 渲染器；
3. 初始化 ImGui 上下文、字体图集、UI 回调绑定；
4. 每帧执行：输入更新 → UI 状态更新 → ImGui 渲染 → 交换链呈现；
5. 处理窗口尺寸变化与点击穿透切换；
6. 程序退出时按逆序释放 UI、ImGui、渲染器、输入与窗口资源。

这套流程里有三个关键设计。第一，初始化阶段一次性完成窗口、输入、渲染和 UI 绑定，运行阶段只做状态推进和渲染输出，主循环更稳，也更容易排查问题。第二，输入更新与渲染放在同一帧闭环，按键触发到画面反馈更快，操作更跟手。第三，退出时按依赖逆序释放资源，避免设备上下文提前销毁带来的悬空调用。

`Application::Tick` 在这里扮演“调度中心”角色：不写业务细节，只负责协调 `InputService`、`OverlayUI` 与 `D3D11Renderer`。后续要加功能（如新 UI 控件或新输入事件）时，通常只改对应模块，不用重写主循环。

4.2 算法与关键公式

输入状态更新模型

- 使用 `unordered_map<keyCode, KeyState>` 管理键状态：`isDown`、`pressedThisFrame`、`releasedThisFrame`、`downTimeSeconds`、`animationProgress`。
- 修饰键采用统一映射策略（例如 Shift 同步到 `VK_SHIFT/VK_LSHIFT/VK_RSHIFT`）。

这个模型的核心是把“瞬时事件”和“持续状态”分开处理。按下/释放事件只在单帧有效，用于触发边沿逻辑；连续状态字段（`isDown`、`downTimeSeconds`、`animationProgress`）负责视觉反馈。帧末调用 `ClearFrameFlags()` 清理边沿标记，避免一次按下触发多帧。该模型的状态流转与稳定性分析见后文“关键问题分析与改进方向”。

动画插值模型

对每个键维护目标强度 $target \in [0,1]$ 与当前强度 x，每帧按速度 v 逼近：

$$\Delta = target - x, \quad step = v \cdot \Delta t.$$

若 $\Delta > step$ 则 $x \leftarrow x + step$；若 $\Delta < -step$ 则 $x \leftarrow x - step$；否则 $x \leftarrow target$。实现里按下和抬起使用不同速度（上升更快、回落略慢），这会让按键反馈更“利落”但不突兀。相比直接在按下时从 0 跳到 1，该插值方式可以显著降低视觉噪声，尤其在高频连按场景中更容易观察输入节奏。

这一步可以写成统一形式：

$$x_{t+\Delta t} = x_t + \mathrm{clamp}(target - x_t,\, -v\Delta t,\, +v\Delta t).$$

用这个写法后，调参更直观：提高 v_{down} 会让按下反馈更快，提高 v_{up} 会让松键衰减更短。参数与 Δt 相乘也让模型具备帧率适配能力，在 60Hz 与 144Hz 下都能维持接近的主观速度。

需要注意的是，极端掉帧时 Δt 可能突增，导致一次更新跨越过大。为避免“亮度跳阶”，可以对 Δt 设上限（例如 1/30s）再参与插值。该策略不会改变正常帧下手感，但能提高突发卡顿场景的观感稳定性。

布局几何计算

布局采用“行-键”结构，每个键带 `widthScale`。行宽近似为：

$$W_{row} = \sum_i (w_{base} \cdot scale \cdot widthScale_i + border) + (n-1) \cdot spacing.$$

窗口尺寸由控制台区与键位区测量值组合得到，并在缩放/预设变化后重算。这里关键不在公式本身，而在“先测量、后渲染”的策略：先根据键位集合计算目标尺寸，再驱动窗口 resize，最后由渲染层按统一尺寸输出。这样能避免控件挤压和裁剪，也能减少“窗口先变、内容后跟上”的跳动感。

窗口尺寸可以写成：

$$W_{win} = \max_r (W_{row,r}) + 2P_x, \qquad H_{win} = H_{console} + \sum_r H_{row,r} + gaps + 2P_y.$$

其中 P_x, P_y 为窗口内边距，$gaps$ 为行间距与区域间距总和。把尺寸模型写清楚后，调试时能更快判断问题出在“行内排布”还是“区域拼接”。

自定义编辑（插入、删除、拖拽换位）本质上是对行-键数组做局部重排。当前规模下复杂度不是瓶颈，真正的风险在状态一致性：结构改动后要同步更新选中键、hover 索引和窗口尺寸缓存，否则容易出现“视觉位置变了，但交互命中还是旧索引”的问题。

表 2: 关键参数与当前实现取值

参数项	当前值	来源代码与作用
按下/抬起插值速度	`rise=12.0`，`fall=10.0`	来自输入更新阶段；控制高亮上升与回落速度。
命令缓冲长度	`24`	命令跟踪模块；限制 `hidehide/showshow` 匹配缓存。
缩放范围	`[1, 4]`（整数步进）	布局缩放函数；限制键位缩放并保持像素风格清晰。
基础键位尺寸	`48x34`，间距 `6`	布局模块常量；用于计算布局几何与窗口目标尺寸。
拖拽期间 Resize 策略	左键按下时暂缓	主循环窗口更新策略；减少拖拽/调参过程中的窗口抖动。

4.3 核心类与模块关系

- `Application`：程序编排、初始化顺序与主循环；
- `Win32Window`：窗口创建、消息分发、拖拽移动、点击穿透；
- `D3D11Renderer`：设备、交换链、渲染目标与 Resize；
- `InputService`：RawInput 解析与按键状态管理；
- `OverlayUI`：控制台面板、键位绘制、交互状态机；
- `KeyboardLayoutPresets`：预设布局与自定义编辑状态管理。

按职责划分，`InputService` 和 `D3D11Renderer` 都是基础能力层，彼此独立；`OverlayUI` 负责消费输入状态并生成绘制内容；`Application` 只做编排，不做业务计算。这样的分层对扩展很友好，例如后续替换键位视觉风格时，大多只需调整 UI 层。

4.4 架构与链路图解

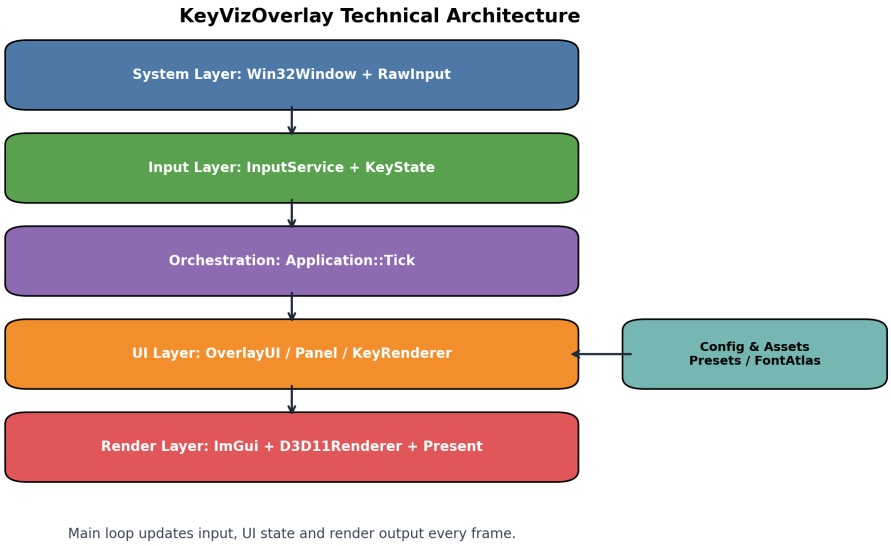

图 1: 系统分层架构与模块交互

图 1 展示了项目的分层结构。上层是 UI 与布局编辑，中间是输入与渲染两条主链路，每帧调度由 `Application::Tick` 完成，下层由窗口能力与配置/资源文件提供支撑。

这套分层有两个直接好处。第一，更好维护：定位问题时先判断属于输入链路、渲染链路还是调度链路，再去对应模块排查。第二，更好替换：例如将来把 D3D11 换成其他后端，只要 UI 绘制接口不变，业务层改动就比较可控。

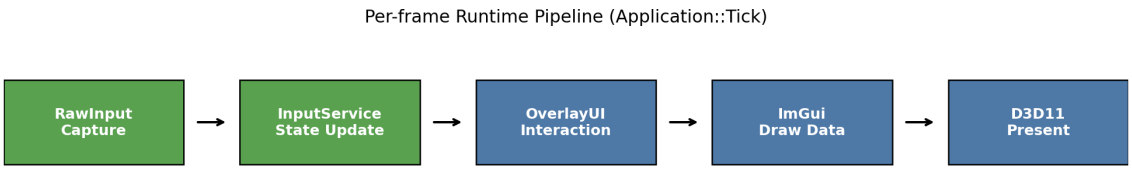

图 2: 每帧执行链路

图 2 对应 `Application::Tick` 的核心调用顺序，输入处理、UI 更新和渲染呈现是串联关系。

这个串联顺序对体验很关键：如果把输入更新放到渲染之后，就会出现“当前帧画的是上一帧状态”，体感会有轻微延迟。现在的顺序保证事件在同一帧内完成“采集-计算-显示”，更符合实时反馈需求。

从模块边界看，当前链路可以概括为三组输入输出关系：`InputService` 输出键状态集合（是否按下、边沿标记、按住时长、插值进度）；`OverlayUI` 消费键状态并生成绘制指令及目标窗口尺寸；`D3D11Renderer` 负责提交绘制数据并完成最终呈现。`Application::Tick` 只负责控制

调用顺序，不承载具体业务规则。

以“按下 W 键并在界面看到高亮”为例，实际链路是：RawInput 产生按键事件 → `InputService` 更新 W 键状态 → `OverlayUI` 按状态计算颜色与亮度 → ImGui 生成 draw data → D3D11 提交并显示。这个例子对应了图 2 的每一段调用。

从时序预算看，单帧耗时可以写成：

$$T_{frame} = T_{input} + T_{ui} + T_{render} + T_{present}.$$

其中 `T_input` 和 `T_ui` 直接影响“按下后多久看到变化”，`T_render` 与 `T_present` 主要影响帧率稳定性。这个拆分便于后续做性能记录：只要在关键节点打点，就能快速定位延迟增长来源。

5 测试与验证

目前测试采用“模块级功能验证 + 场景回归”的方式，重点覆盖输入、渲染和配置链路。

表 3: 测试案例与结果汇总

编号	输入/操作	预期输出	结果	验证目的
T1	按下/释放普通键（如 W）	键帽高亮、抬起后渐隐，`pressed/released` 标志仅一帧有效	通过	验证输入状态机与动画更新
T2	按下左/右 Shift 或 Ctrl	对应修饰键统一显示激活状态	通过	验证修饰键别名映射
T3	连续输入 `hidehide` 与 `showshow`	控制台隐藏/恢复，键位窗口位置连续无跳变	通过	验证命令跟踪与窗口重定位
T4	切换布局预设（键盘、键盘 + 鼠标、自定义）	行集变化正确，窗口尺寸自适应重算	通过	验证布局上下文重建
T5	自定义编辑：拖拽换位、右键删除、增删行	键位结构正确更新，界面无异常跳变	通过	验证编辑命令与布局更新

从测试覆盖看，T1-T3 主要验证输入链路正确性，T4-T5 主要验证布局与交互稳定性。这样分组后，输入相关改动优先回归 T1-T3，UI 布局和编辑逻辑改动优先回归 T4-T5，能减少无效测试时间。

项目在测试时密切关注“状态同步时机”。例如控制台隐藏/显示与点击穿透切换涉及窗口样式修改，如果处理顺序不当，容易出现短暂不可交互或位置突变。把这类场景单独列为测试项，可以在功能迭代时尽早暴露回归问题。

表 4: 高风险点与观测指标

风险点	观测指标	建议阈值/现象
高频连按时反馈拖尾	按下到高亮可见的帧延迟	绝大多数输入在 1 帧内可见变化
预设切换导致窗口抖动	切换后窗口位置与尺寸稳定时间	1-2 帧内收敛，无明显二次跳变
编辑后命中区域错位	鼠标点击目标与实际选中项一致性	不出现“点 A 选中 B”现象

6 关键问题分析与改进方向

6.1 输入状态与视觉反馈的一致性

实现里最关键的一点，是把输入事件和视觉反馈稳定地绑定在同一帧内。RawInput 到来后，系统要同时处理两类状态：一类是 `pressedThisFrame/releasedThisFrame` 这类单帧边沿信号，另一类是 `isDown/downTimeSeconds/animationProgress` 这类连续状态。两者如果混在一起更新，很容易出现“触发一次却连续多帧响应”或“抬起后仍保持高亮”的问题。

实现上采用了“事件边沿 + 持续状态”分离策略，并在帧末调用 `ClearFrameFlags()` 清理边沿标志。这样能保证命令识别、发光动画和按住时长统计各自稳定，互不干扰。

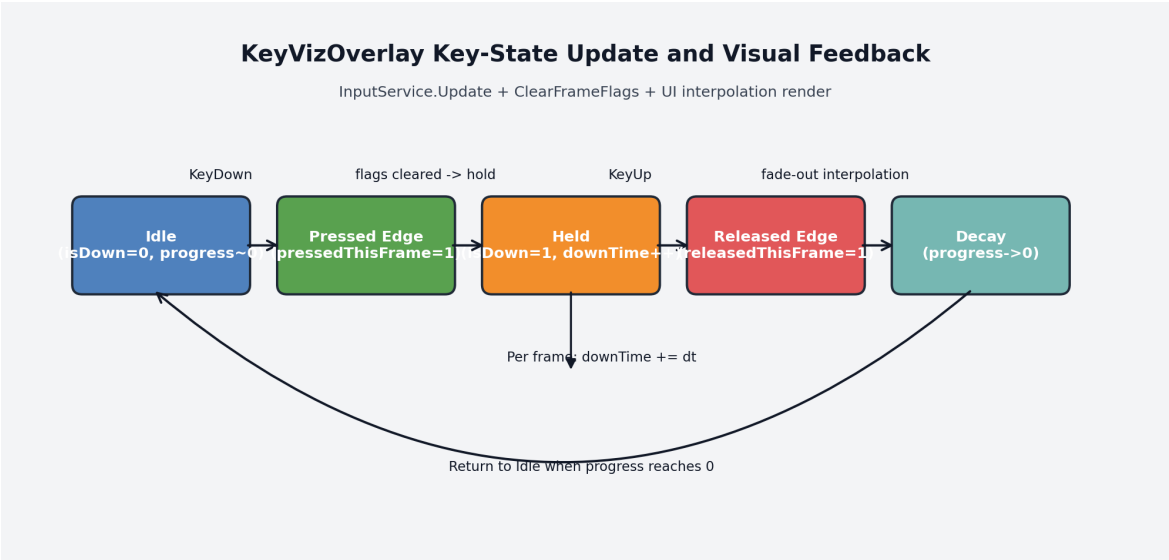

图 3: 按键状态更新与视觉反馈状态机

图 3 展示了按键从按下到抬起再回到空闲的完整路径。这个状态机把关键更新点明确下来：KeyDown 只负责触发边沿，Held 负责累计按住时长，Released 后进入衰减插值，最后在亮度回到 0 时回到 Idle。

6.2 交互模式切换的稳定性

另一个关键点是窗口交互模式切换。项目需要在“可拖拽可交互”和“点击穿透”两种模式之间切换：控制台显示时要能操作控件，控制台隐藏后又不能遮挡前台应用。这里如果切换时机处理不好，很容易出现窗口瞬间不可用或位置跳变。

当前实现把模式切换放在主循环统一更新阶段，并配合“拖拽期间暂缓 resize”的策略，减少频繁重设窗口带来的抖动。结合前面的测试项（特别是 T3/T4/T5），这部分逻辑已经能覆盖常见回归风险。

6.3 开发过程中的问题复盘

表 5: 开发过程关键问题复盘

问题现象	定位结论	处理方式	结果
左右修饰键显示不一致（如按下左 Shift 时部分键位未同步高亮）	Shift 主键与左右键码存在别名差异	在输入层统一修饰键映射	修饰键显示一致，命令识别稳定
拖拽窗口或调节参数时出现抖动	交互过程中窗口尺寸高频重设，造成连续 Resize	在主循环中增加“左键按下时暂缓重设”策略	交互过程更平滑，窗口跳变明显减少

6.4 改进方向

- 增加性能打点：记录 `T_input/T_ui/T_render/T_present` 的分段耗时分布，定位高频输入场景下的抖动来源。
- 补充自动化回放：将预设切换、拖拽换位和命令输入串成回放序列，降低重复手工回归成本。
- 扩展主题参数：在保持可读性的前提下开放颜色、描边和阴影强度配置，提升不同直播/录屏背景下的适配性。

7 收获

- 通过分层设计把窗口、渲染、输入、UI 解耦后，后续功能迭代（如自定义预设）更顺畅；
- 对 RawInput 与实时渲染协作有了更清晰的工程认识；
- 在迭代中逐步形成了“先可用、再修边界与交互细节”的开发节奏。

8 AI 工具使用披露

- 工具名称与版本：OpenAI Codex（GPT-5 系列）
- 用途：代码实现与重构建议、边界条件处理、报告整理与图示脚本辅助
- 代码部分使用过 AI 辅助开发。
- 使用位置（代码示例）：`KeyVizOverlay/src/input/InputService.cpp`（修饰键映射与动画插值更新）；`KeyVizOverlay/src/ui/OverlayConsoleCommandTracker.cpp`（命令缓冲与匹配）；`KeyVizOverlay/src/ui/KeyboardLayoutPresets.cpp`（布局持久化与预设管理）；`KeyVizOverlay/src/app/Application.cpp`（初始化流程与主循环调度）。

附录：须提交链接

- 代码仓库地址：https://github.com/yirenzhang/KeyVizOverlay.git
- Bilibili 视频: BV1d4X7BGE9K